

IND320 Project Work

- **Github Link** : <https://github.com/Mobashra/M-Abeer-Project>
- **Streamlit Link** : <https://m-abeer-project.streamlit.app/>

Project Log

In this final phase of the project, the focus shifted from basic exploration to building a robust, interactive data product. The work involved significant backend data engineering in the Jupyter Notebook and a complete architectural refactoring of the Streamlit application.

Data Engineering & Backend I expanded the dataset by retrieving hourly production data and consumption data for 2021–2024 via the Elhub API. To handle the volume, I utilized the same month-by-month fetching strategy from Part 2. These data were appended to the existing MongoDB and Cassandra clusters. For Cassandra, I created a new `consumption_data` and `production_data` table with an appropriate schema.

Streamlit Refactoring & Visualization Leveraging "Artistic Freedom," I reorganized the app structure into three logical modules: **Explorative**, **Diagnostic**, and **Predictive**. A custom CSS theme (Teal/Amber/Indigo) was implemented to visually distinguish these sections. A major task was the migration of all static Matplotlib/Seaborn plots to dynamic **Plotly** visualizations, adding interactivity to line charts, spectrograms, and polar wind roses.

New Analytical Features

- **Map Selector:** I implemented an interactive Mapbox visualization using GeoJSON overlays. The map features dynamic layer switching: displaying Price Areas (NO1–NO5) at normal view and detailed Municipalities when click the `show municipalities` option with the names displayed while hovering. I implemented logic to capture click events, storing the selected coordinate and region in the session state to persist context across all analysis pages.
- **Snow Drift Analysis:** Adapting the Tabler (2003) physics model, I built a calculator for annual and monthly snow

transport ($Q_{t\$}$). The module includes configurable parameters (Fetch, Transport Distance) and visualizes the results via a dual-tab interface: one for Annual totals and one for a **Monthly Breakdown** (Bonus task), alongside a wind rose showing drift-causing wind sectors.

- **Correlations:** The sliding window correlation page was enhanced with selectors for specific weather variables and energy groups. Testing revealed that wind speed correlations are strongest in winter, while temperature correlations peak during the heating season (Oct–March).
- **Forecasting (SARIMAX):** I built a comprehensive forecasting interface allowing users to configure ARIMA parameters (p, d, q, P, D, Q) and horizon. Crucially, I implemented **Exogenous Variables** (Bonus task), allowing the model to ingest real-time weather data (e.g., Temperature) to improve forecast accuracy.

Bonus Content Implementation I prioritized user experience by implementing several bonus tasks. **Error Handling** was centralized; attempting to run analyses without a selected location now triggers a helpful warning with a direct link to the map. **Waiting Indicators** (spinners) provide feedback during API calls. Finally, the **Monthly Snow Drift** and **Exogenous Weather Forecasting** provide deeper analytical capabilities beyond the base requirements.

AI Usage

AI Tools Used: GitHub Copilot / Gemini Pro

Throughout Part 4, AI assistance was utilized as to accelerate development in the following key areas:

- **GeoJSON & Mapping Logic:** AI assisted in processing Norwegian GeoJSON files, specifically implementing the logic to switch layers dynamically between Price Areas vs. Municipalities and handling "point-in-polygon" detection to map specific coordinates to their parent Price Area.
- **Visualization Migration:** AI provided guidance on converting static Matplotlib code into interactive Plotly graph objects. This was particularly helpful for complex charts like the **Polar Wind Rose** (for snow drift) and the **Spectrograms** (for signal processing).
- **Mathematical Implementation:** For the Snow Drift module, AI helped verify the translation of the Tabler (2003) mathematical formulas into vectorized Python functions. It also assisted in structuring the `analysis_functions.py` file to keep the main Streamlit logic clean.

- **SARIMAX Modeling:** AI assisted in implementing the `statsmodels` SARIMAX class, specifically regarding the handling of **Exogenous Variables**. It helped write the code to align the timestamps of weather data (features) with energy data (target) to ensure the model had the correct dimensions for the forecast horizon.
- **Error Handling & CSS:** I used AI to generate the custom CSS wrappers for the sidebar styling and to write the robust `check_session_state` function that handles missing data connections or unselected regions gracefully with user-friendly error messages.

Importing all necessary libraries

```
In [ ]: import os
import time
import sys
import json
from datetime import datetime, timedelta
from typing import List, Dict
from zoneinfo import ZoneInfo

import requests
import pandas as pd
import plotly.express as px
from dotenv import load_dotenv
from pymongo import MongoClient
from pyspark.sql import SparkSession
from pyspark.sql.functions import sum as spark_sum, month
from pyspark.sql.functions import col
```

Data Retrieval and Processing Pipeline

- **Methodology:** To circumvent API timeouts and rate limits, a looping mechanism was engineered to fetch data in monthly increments. The script iterates through years (2021–2024) and months (1–12), constructing precise ISO-formatted timestamps for the start and end of each period.
- **Data Handling:** A generic fetching function (`fetch_elhub_data`) was created to handle JSON parsing and key extraction dynamically.

- **Timezone Standardization:** All timestamps (`startTime` , `endTime` , `lastUpdatedTime`) were converted to the `Europe/Oslo` timezone to ensure consistency across the dataset.
- **Persistence:** The raw data for both production and consumption was serialized into CSV files (`elhub_production_2021_2024.csv` and `elhub_consumption_2021_2024.csv`) to serve as an intermediate storage layer.

```
In [ ]: # --- 1. CORE API FUNCTION (Generic) ---
def fetch_elhub_data(dataset_name: str, start_date: str, end_date: str) -> List[Dict]:
    base_url = "https://api.elhub.no/energy-data/v0/price-areas"
    params = {'dataset': dataset_name, 'startDate': start_date, 'endDate': end_date}

    # Convert UPPER_CASE dataset name to camelCase key for JSON parsing
    # e.g. PRODUCTION_PER_GROUP_MBA_HOUR -> productionPerGroupMbaHour
    tokens = dataset_name.lower().split('_')
    json_key = tokens[0] + ''.join(x.title() for x in tokens[1:])

    try:
        response = requests.get(base_url, params=params, timeout=30)
        response.raise_for_status()
        data = response.json()

        all_records = []
        if 'data' in data:
            for price_area_data in data['data']:
                if 'attributes' in price_area_data and json_key in price_area_data['attributes']:
                    all_records.extend(price_area_data['attributes'][json_key])

        return all_records

    except requests.exceptions.RequestException as e:
        # If a specific month fails, we print error but don't crash the whole script
        print(f" [!] Error fetching {start_date[:7]}: {e}")
        return []

# --- 2. LOOPING FUNCTION (Years + Months) ---
def fetch_data_monthly_loop(dataset_name: str, start_year: int, end_year: int) -> pd.DataFrame:
```

```

all_records = []

print(f"Starting retrieval for: {dataset_name}")
print(f"Period: {start_year} - {end_year}")

# Loop through every year requested
for year in range(start_year, end_year + 1):
    print(f"\n Processing Year: {year}")

    # Loop through every month (Safety against 400 Bad Request)
    for month in range(1, 13):

        # Calculate start and end of the month
        month_start = datetime(year, month, 1, 0, 0, 0)

        # Logic for rollover (Dec -> Jan)
        if month == 12:
            month_end = datetime(year + 1, 1, 1, 0, 0, 0)
        else:
            month_end = datetime(year, month + 1, 1, 0, 0, 0)

        # Format for API
        start_str = month_start.strftime('%Y-%m-%dT%H:%M:%S+01:00')
        end_str = month_end.strftime('%Y-%m-%dT%H:%M:%S+01:00')

        # Fetch
        records = fetch_elhub_data(dataset_name, start_str, end_str)

        if records:
            all_records.extend(records)
            print(f"    - {month_start.strftime('%B')}: Retrieved {len(records)} records")
        else:
            print(f"    - {month_start.strftime('%B')}: No data")

        # Sleep to avoid "429 Too Many Requests"
        time.sleep(0.2)

```

```

# Convert to DataFrame
df = pd.DataFrame(all_records)

# Fix Timezones
if not df.empty:
    print(f"\n Formatting timezones for {len(df)} rows...")
    cols = ['startTime', 'endTime', 'lastUpdatedTime']
    for col in cols:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], utc=True).dt.tz_convert("Europe/Oslo")

    return df

# --- 3. MAIN EXECUTION ---
if __name__ == "__main__":

    # --- PART 1: PRODUCTION (2021 - 2024) ---
    print(">>> TASK 1: FETCHING PRODUCTION DATA (2021-2024)")
    df_prod = fetch_data_monthly_loop(
        dataset_name="PRODUCTION_PER_GROUP_MBA_HOUR",
        start_year=2021,
        end_year=2024
    )

    if not df_prod.empty:
        filename = "elhub_production_2021_2024.csv"
        df_prod.to_csv(filename, index=False)
        print(f"SUCCESS: Saved {filename}")

    print("-" * 40)

    # --- PART 2: CONSUMPTION (2021 - 2024) ---
    print(">>> TASK 2: FETCHING CONSUMPTION DATA (2021-2024)")
    df_cons = fetch_data_monthly_loop(
        dataset_name="CONSUMPTION_PER_GROUP_MBA_HOUR",
        start_year=2021,
        end_year=2024
    )

```

```
if not df_cons.empty:
    filename = "elhub_consumption_2021_2024.csv"
    df_cons.to_csv(filename, index=False)
    print(f"SUCCESS: Saved {filename}")

print("\nAll operations complete.")
```

```
>>> TASK 1: FETCHING PRODUCTION DATA (2021-2024)
Starting retrieval for: PRODUCTION_PER_GROUP_MBA_HOUR
Period: 2021 - 2024
```

Processing Year: 2021

- January: Retrieved 17856 records
- February: Retrieved 16128 records
- March: Retrieved 17832 records
- April: Retrieved 17280 records
- May: Retrieved 17856 records
- June: Retrieved 17976 records
- July: Retrieved 18600 records
- August: Retrieved 18600 records
- September: Retrieved 18000 records
- October: Retrieved 18625 records
- November: Retrieved 18000 records
- December: Retrieved 18600 records

Processing Year: 2022

- January: Retrieved 18600 records
- February: Retrieved 16800 records
- March: Retrieved 18575 records
- April: Retrieved 18000 records
- May: Retrieved 18600 records
- June: Retrieved 18000 records
- July: Retrieved 18600 records
- August: Retrieved 18600 records
- September: Retrieved 18000 records
- October: Retrieved 18625 records
- November: Retrieved 18000 records

- December: Retrieved 18600 records

Processing Year: 2023

- January: Retrieved 18600 records
- February: Retrieved 16800 records
- March: Retrieved 18575 records
- April: Retrieved 18000 records
- May: Retrieved 18600 records
- June: Retrieved 18000 records
- July: Retrieved 18600 records
- August: Retrieved 18600 records
- September: Retrieved 18000 records
- October: Retrieved 18625 records
- November: Retrieved 18000 records
- December: Retrieved 18600 records

Processing Year: 2024

- January: Retrieved 18600 records
- February: Retrieved 17400 records
- March: Retrieved 18575 records
- April: Retrieved 18000 records
- May: Retrieved 18600 records
- June: Retrieved 18000 records
- July: Retrieved 18600 records
- August: Retrieved 18600 records
- September: Retrieved 18000 records
- October: Retrieved 18625 records
- November: Retrieved 18000 records
- December: Retrieved 18600 records

Formatting timezones for 872953 rows...

SUCCESS: Saved elhub_production_2021_2024.csv

>>> TASK 2: FETCHING CONSUMPTION DATA (2021-2024)

Starting retrieval for: CONSUMPTION_PER_GROUP_MBA_HOUR

Period: 2021 - 2024

Processing Year: 2021

- January: Retrieved 18600 records
- February: Retrieved 16800 records
- March: Retrieved 18575 records
- April: Retrieved 18000 records
- May: Retrieved 18600 records
- June: Retrieved 18000 records
- July: Retrieved 18600 records
- August: Retrieved 18600 records
- September: Retrieved 18000 records
- October: Retrieved 18625 records
- November: Retrieved 18000 records
- December: Retrieved 18600 records

Processing Year: 2022

- January: Retrieved 18600 records
- February: Retrieved 16800 records
- March: Retrieved 18575 records
- April: Retrieved 18000 records
- May: Retrieved 18600 records
- June: Retrieved 18000 records
- July: Retrieved 18600 records
- August: Retrieved 18600 records
- September: Retrieved 18000 records
- October: Retrieved 18625 records
- November: Retrieved 18000 records
- December: Retrieved 18600 records

Processing Year: 2023

- January: Retrieved 18600 records
- February: Retrieved 16800 records
- March: Retrieved 18575 records
- April: Retrieved 18000 records
- May: Retrieved 18600 records
- June: Retrieved 18000 records
- July: Retrieved 18600 records
- August: Retrieved 18600 records
- September: Retrieved 18000 records
- October: Retrieved 18625 records

- November: Retrieved 18000 records
- December: Retrieved 18600 records

Processing Year: 2024

- January: Retrieved 18600 records
- February: Retrieved 17400 records
- March: Retrieved 18575 records
- April: Retrieved 18000 records
- May: Retrieved 18600 records
- June: Retrieved 18000 records
- July: Retrieved 18600 records
- August: Retrieved 18600 records
- September: Retrieved 18000 records
- October: Retrieved 18625 records
- November: Retrieved 18000 records
- December: Retrieved 18600 records

Formatting timezones for 876600 rows...

SUCCESS: Saved elhub_consumption_2021_2024.csv

All operations complete.

Spark Data Processing & Database Ingestion

This script executes the ETL (Extract, Transform, Load) pipeline to populate the database layer:

- **Spark Initialization:** Configures a `SparkSession` with the `spark-cassandra-connector` and specific tuning parameters for local optimization.
- **Schema Normalization:** Reads the raw CSV data and standardizes column names from the API's CamelCase format (e.g., `productionGroup`) to the database's snake_case format (e.g., `production_group`).
- **Dual-Database Upload:**
 - **Cassandra:** Uses the Spark connector to append processed data to the `production_data` and `consumption_data` tables.
 - **MongoDB:** Uses `pymongo` to insert documents, explicitly converting timestamp strings into Python `datetime` objects to ensure correct `ISODate` storage.

```

In [ ]: os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable
# Load environment variables from .env file
load_dotenv()

# --- CONFIGURATION ---
CASSANDRA_KEYSPACE = os.getenv("CASSANDRA_KEYSPACE", "energy_data")

# MongoDB Config
MONGO_USER = os.getenv("MONGO_USER")
MONGO_PASS = os.getenv("MONGO_PASS")
MONGO_CLUSTER = os.getenv("MONGO_CLUSTER")
MONGO_DB = os.getenv("MONGO_DB")

# Collection Names from .env
COL_PROD = os.getenv("MONGO_COLLECTION_PROD", "production_mba_hour")
COL_CONS = os.getenv("MONGO_COLLECTION_CONS", "consumption_mba_hour")

def get_spark_session():
    """
    Initializes Spark with the optimized Cassandra configuration.
    """
    return (
        SparkSession.builder
        .appName("ElhubSparkCassandra")
        .config("spark.jars.packages", "com.datastax.spark:spark-cassandra-connector_2.12:3.5.1")
        .config("spark.cassandra.connection.host", "localhost")
        .config("spark.cassandra.connection.port", "9042")
        .config("spark.sql.extensions", "com.datastax.spark.connector.CassandraSparkExtensions")
        .config("spark.sql.catalog.casscatalog", "com.datastax.spark.connector.datasource.CassandraCatalog")
        .config("spark.sql.shuffle.partitions", "16")
        .config("spark.default.parallelism", "16")
        .config("spark.cassandra.output.concurrent.writes", "10")
        .config("spark.cassandra.output.batch.size.bytes", 1024)
        .getOrCreate()
    )

```

```

def process_and_upload(spark, csv_path, cassandra_table, mongo_collection_name, data_type):
    """
    Reads CSV, transforms columns to match Schema, and uploads to Cassandra & MongoDB.
    """
    print(f"\n" + "="*50)
    print(f"PROCESSING: {data_type.upper()}")
    print(f"Source: {csv_path}")
    print(f"Target Cassandra Table: {cassandra_table}")
    print(f"Target MongoDB Collection: {mongo_collection_name}")
    print("="*50)

    # 1. READ CSV (Using Pandas for safe date parsing)
    try:
        pdf = pd.read_csv(csv_path)
        print(f"--> Loaded CSV. Shape: {pdf.shape}")
    except FileNotFoundError:
        print(f"ERROR: File {csv_path} not found. Skipping.")
        return

    # 2. CREATE SPARK DATAFRAME
    spark_df = spark.createDataFrame(pdf)

    # 3. RENAME COLUMNS (CSV Header -> Database Column Name)
    rename_mapping = {
        "priceArea": "price_area",
        "startTime": "start_time",
        "endTime": "end_time",
        "lastUpdatedTime": "last_updated_time",
        "quantityKwh": "value"
    }

    # Specific mapping based on data type
    if data_type == "production":
        rename_mapping["productionGroup"] = "production_group"

    elif data_type == "consumption":
        rename_mapping["consumptionGroup"] = "consumption_group"
        rename_mapping["meteringPointCount"] = "metering_point_count"

```

```

# Apply renaming logic
for old_name, new_name in rename_mapping.items():
    if old_name in spark_df.columns:
        spark_df = spark_df.withColumnRenamed(old_name, new_name)

# 4. WRITE TO CASSANDRA (OVERWRITE MODE WITH TRUNCATE CONFIRMATION)
mode = "append"

print(f"-> Writing to Cassandra (Keyspace: {CASSANDRA_KEYSPACE}, Mode: {mode})...")

# Safety: Ensure we don't try to write 'metering_point_count' to the Production table
if data_type == "production" and "metering_point_count" in spark_df.columns:
    spark_df = spark_df.drop("metering_point_count")

try:
    spark_df.write \
        .format("org.apache.spark.sql.cassandra") \
        .mode(mode) \
        .option("confirm.truncate", "true") \
        .option("keyspace", CASSANDRA_KEYSPACE) \
        .option("table", cassandra_table) \
        .save()
    print("-> Cassandra Write Success!")
except Exception as e:
    print(f"-> Cassandra Error: {e}")
    print("    (Check if the table exists in cqlsh!)")

# 5. WRITE TO MONGODB (CLEAN SLATE)
print(f"-> Writing to MongoDB ({mongo_collection_name})...")

uri = f"mongodb+srv://{MONGO_USER}:{MONGO_PASS}@{MONGO_CLUSTER}"
try:
    client = MongoClient(uri)
    db = client[MONGO_DB]
    collection = db[mongo_collection_name]

```

```

# Prepare data (reuse Pandas DF)
pdf_renamed = pdf.rename(columns=rename_mapping)

# CRITICAL: Convert Dates to Python Datetime for MongoDB
# This ensures dates are stored as ISODate objects, not Strings
if "start_time" in pdf_renamed.columns:
    pdf_renamed['start_time'] = pd.to_datetime(pdf_renamed['start_time'], utc=True)
if "end_time" in pdf_renamed.columns:
    pdf_renamed['end_time'] = pd.to_datetime(pdf_renamed['end_time'], utc=True)
if "last_updated_time" in pdf_renamed.columns:
    pdf_renamed['last_updated_time'] = pd.to_datetime(pdf_renamed['last_updated_time'], utc=True)

records = pdf_renamed.to_dict("records")

if records:
    collection.insert_many(records)
    print(f"-> MongoDB Write Success! ({len(records)} docs)")

client.close()
except Exception as e:
    print(f"-> MongoDB Error: {e}")

# --- MAIN EXECUTION ---
if __name__ == "__main__":

    # Initialize Spark
    spark = get_spark_session()
    print("Spark Session Active.")

    # --- TASK 1: PRODUCTION (FULL HISTORY 2021-2024) ---
    process_and_upload(
        spark=spark,
        csv_path="elhub_production_2021_2024.csv",
        cassandra_table="production_data",
        mongo_collection_name=COL_PROD,
        data_type="production"
    )

```

```

# --- TASK 2: CONSUMPTION (FULL HISTORY 2021-2024) ---
process_and_upload(
    spark=spark,
    csv_path="elhub_consumption_2021_2024.csv",
    cassandra_table="consumption_data",
    mongo_collection_name=COL_CONS,
    data_type="consumption"
)

# Stop Spark
spark.stop()
print("\nPipeline finished successfully.")

```

25/11/25 23:13:02 WARN Utils: Your hostname, Mobashras-MacBook-Air.local resolves to a loopback address: 127.0.0.1; using 192.168.11.23 instead (on interface en0)

25/11/25 23:13:02 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
 :: loading settings :: url = jar:file:/Users/mobashraabeer/miniconda3/envs/D2D_env/lib/python3.12/site-packages/pyspark/jars/ivy-2.5.1.jar!/org/apache/ivy/core/settings/ivysettings.xml

Ivy Default Cache set to: /Users/mobashraabeer/.ivy2/cache

The jars for the packages stored in: /Users/mobashraabeer/.ivy2/jars

com.datastax.spark#spark-cassandra-connector_2.12 added as a dependency

:: resolving dependencies :: org.apache.spark#spark-submit-parent-8c5d4d65-4a1c-4bd9-a02c-dd49bcb666c8;1.0

confs: [default]

found com.datastax.spark#spark-cassandra-connector_2.12;3.5.1 in central

found com.datastax.spark#spark-cassandra-connector-driver_2.12;3.5.1 in central

found org.scala-lang.modules#scala-collection-compat_2.12;2.11.0 in central

found org.apache.cassandra#java-driver-core-shaded;4.18.1 in central

found com.datastax.oss#native-protocol;1.5.1 in central

found com.datastax.oss#java-driver-shaded-guava;25.1-jre-graal-sub-1 in central

found com.typesafe#config;1.4.1 in central

found org.slf4j#slf4j-api;1.7.26 in central

found io.dropwizard.metrics#metrics-core;4.1.18 in central

found org.hdrhistogram#HdrHistogram;2.1.12 in central

found org.reactivestreams#reactive-streams;1.0.3 in central

found org.apache.cassandra#java-driver-mapper-runtime;4.18.1 in central

found org.apache.cassandra#java-driver-query-builder;4.18.1 in central

found org.apache.commons#commons-lang3;3.10 in central

```

found com.thoughtworks.paranamer#paranamer;2.8 in central
found org.scala-lang#scala-reflect;2.12.19 in central
:: resolution report :: resolve 187ms :: artifacts dl 6ms
:: modules in use:
com.datastax.oss#java-driver-shaded-guava;25.1-jre-graal-sub-1 from central in [default]
com.datastax.oss#native-protocol;1.5.1 from central in [default]
com.datastax.spark#spark-cassandra-connector-driver_2.12;3.5.1 from central in [default]
com.datastax.spark#spark-cassandra-connector_2.12;3.5.1 from central in [default]
com.thoughtworks.paranamer#paranamer;2.8 from central in [default]
com.typesafe#config;1.4.1 from central in [default]
io.dropwizard.metrics#metrics-core;4.1.18 from central in [default]
org.apache.cassandra#java-driver-core-shaded;4.18.1 from central in [default]
org.apache.cassandra#java-driver-mapper-runtime;4.18.1 from central in [default]
org.apache.cassandra#java-driver-query-builder;4.18.1 from central in [default]
org.apache.commons#commons-lang3;3.10 from central in [default]
org.hdrhistogram#HdrHistogram;2.1.12 from central in [default]
org.reactivestreams#reactive-streams;1.0.3 from central in [default]
org.scala-lang#scala-reflect;2.12.19 from central in [default]
org.scala-lang.modules#scala-collection-compat_2.12;2.11.0 from central in [default]
org.slf4j#slf4j-api;1.7.26 from central in [default]

```

	modules						artifacts		
	conf	number	search	downloaded	evicted		number	downloaded	
	default	16	0	0	0		16	0	

```

:: retrieving :: org.apache.spark#spark-submit-parent-8c5d4d65-4a1c-4bd9-a02c-dd49bcb666c8
  confs: [default]
  0 artifacts copied, 16 already retrieved (0kB/4ms)

```

```

25/11/25 23:13:02 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using bu
iltin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).

```


Spark Session Active.

=====

PROCESSING: PRODUCTION

Source: elhub_production_2021_2024.csv

Target Cassandra Table: production_data

Target MongoDB Collection: production_mba_hour

=====

-> Loaded CSV. Shape: (872953, 6)

-> Writing to Cassandra (Keyspace: energy_data, Mode: append)...

25/11/25 23:13:18 WARN TaskSetManager: Stage 0 contains a task of very large size (3332 KiB). The maximum recommended task size is 1000 KiB.

-> Cassandra Write Success!

-> Writing to MongoDB (production_mba_hour)...

-> MongoDB Write Success! (872953 docs)

=====

PROCESSING: CONSUMPTION

Source: elhub_consumption_2021_2024.csv

Target Cassandra Table: consumption_data

Target MongoDB Collection: consumption_mba_hour

=====

-> Loaded CSV. Shape: (876600, 7)

-> Writing to Cassandra (Keyspace: energy_data, Mode: append)...

25/11/25 23:42:54 WARN TaskSetManager: Stage 1 contains a task of very large size (3534 KiB). The maximum recommended task size is 1000 KiB.

-> Cassandra Write Success!

-> Writing to MongoDB (consumption_mba_hour)...

-> MongoDB Write Success! (876600 docs)

Pipeline finished successfully.